TALLER DE METODOLOGÍAS ÁGILES · 2026-2

Asesoría y propuesta de escenarios

Qué pidió el profe, qué leer, con qué grabar y qué escenarios y roles probar en los ciclos

PAREJA

Ana Sofía Arango Yanza · Juan José Arias Chacua

FECHA Y VERSIÓN

06/11/2026 · v1 · propuesta para decidir en pareja

1 Qué pidió el profe y qué proponemos

Son seis pedidos. Ninguno cambia la práctica que medimos (desarrollar una tarea en pair programming) ni las cinco métricas: cambian la forma de grabar y las configuraciones que probamos en los ciclos.

Pedido	Propuesta	Sección
Leer la fuente principal del pair programming y entender de dónde viene y para qué sirve	Una ruta de lectura corta, con un mapa para llenar entre los dos	2
Una herramienta específica de pair programming para grabar	VS Code Live Share (o Code With Me si usan PyCharm), sin dejar de grabar pantalla y audio	3
Justificar cuántos casos tiene la línea base	5 casos: los 3 que ya tienen (uno por tamaño) y 2 más grabados con la herramienta	4
Roles definidos, también para los agentes, y agentes con seniority	Una tarjeta por rol: quién escribe, quién revisa y quién decide. La seniority se aplica a los agentes	5
Explorar configuraciones: agente y persona en las dos direcciones, dos personas más agente, varios agentes	4 escenarios aparte de la línea base, con 3 casos cada uno: 17 casos en total	6
Reducir la erosión del pensamiento crítico	Cómo se mide con lo que ya registran y cinco reglas para reducirla	7

ANTES DE SEGUIR

El enunciado pide al menos diez casos en la línea base. El profe dijo en la asesoría que la cantidad es flexible si se justifica. Conviene dejarlo escrito en un acta y citarlo en el reporte, para que nadie lo lea como un incumplimiento.

2 Qué leer para entender el tema

El libro de referencia es *Pair Programming Illuminated* [2]. Si no lo consiguen completo, el artículo de los mismos autores [3] es corto, gratuito y cuenta lo esencial.

#	Qué leer	Qué responde	Quién
1	Beck, *Extreme Programming Explained* [1]: la parte de prácticas	De dónde viene: el pair programming nace como práctica de XP, con todo el código de producción escrito en pareja	Los dos

2	Williams et al. (2000) [3] y, si se consigue, el libro [2]	Para qué sirve: menos defectos, mejor diseño y más confianza a cambio de más esfuerzo. Aquí aparecen los roles de conductor y navegante	Ana Sofía
3	Hannay et al. (2009) [4], el metaanálisis	Qué dice la evidencia: en tareas simples la pareja va más rápido; en complejas, gana calidad pero gasta más esfuerzo. Por eso medimos por tamaño	Juan José
4	Bryant et al. (2008) [6] y Chong y Hurlbutt (2007) [7]	Los roles en la práctica: en parejas reales, el navegante no hace solo de "estratega" y los dos trabajan al mismo nivel. Lo que más pesa es quién sabe más	Ana Sofía
5	Plonka et al. (2015) [8]	Transferencia de conocimiento: cómo el que sabe más le enseña al otro mientras programan. Es lo que medimos con el codewalk	Juan José
6	Lyu et al. (2025) [13] y Daryanto et al. (2026) [14]	Qué cambia con la IA: con IA se pregunta por sintaxis y conceptos, y con la pareja se discuten ideas. Con un compañero presente se copia menos código de la IA	Los dos

El mapa: cada uno llena estas siete filas con lo que leyó, en una frase y con su fuente: **origen**, **objetivo**, **roles**, **beneficios**, **costos**, **cuándo conviene y cuándo no**, y **qué cambia con la IA**. Después se intercambian y explican la parte del otro, igual que en el codewalk. Ese mapa sirve también para la exposición.

3 Herramienta para grabar

OBS graba la pantalla, pero no es una herramienta de pair programming. La recomendación es programar en una sesión compartida, hecha para esto, y seguir grabando pantalla y audio.

Herramienta	Qué aporta	Ojo con
VS Code Live Share [20] (recomendada)	Los dos trabajan en el mismo código, cada uno desde su computador. Muestra el cursor y el nombre de cada uno, y el navegante puede seguir al conductor o señalar una línea. Es gratuita y oficial de Microsoft	Microsoft la tiene en mantenimiento: funciona, pero no le agregan funciones. No graba sola
Code With Me (JetBrains) [21]	Lo mismo dentro de PyCharm, con un modo explícito para pair programming. Viene incluida en PyCharm Community	La versión gratuita limita invitados y duración de la sesión. No graba sola
Grabación de pantalla, audio y cámara	Es lo que usan los estudios de pair programming que analizan sesiones reales [8][10][11]. OBS sirve para esto	Debe grabar la pantalla de quien conduce
CodeWatcher [22]	Registra cada edición en VS Code con hora. Daría datos finos de quién escribió qué	Es un preprint sin revisión por pares. Solo si sobra tiempo

QUÉ NO CAMBIA

La ficha, las métricas y los commits son los mismos. Solo cambian dos cosas: quien conduce abre la sesión de Live Share y el navegante se une, y se agrega una regla, **solo el conductor escribe**; el navegante puede seguir y señalar, pero no editar. Así los roles se ven en el video.

4 La línea base: por qué solo en pareja y cuántos casos

La línea base es la práctica como se hace hoy, sin IA: dos personas, una conduce y la otra navega. Si la línea base tuviera agente, no habría un "antes" con el cual comparar.

El proyecto pide medir cómo cambia la práctica cuando entra el agente, así que primero hay que saber cómo funciona sin él. Por eso la línea base es solo de ustedes dos, y el agente aparece en los ciclos.

YA MEDIDOS	CON LA HERRAMIENTA	TOTAL
3	**+2**	**5**
t06 (S), t03 (M) y t12 (L): uno por tamaño	t15 (S) y t07 (M), los siguientes en el orden del sorteo	2 S · 2 M · 1 L

Cómo justificar 5:

1. **Todos los tamaños.** El tamaño cambia el efecto del pair programming [4][5], así que la línea base tiene que cubrir S, M y L. Los 3 primeros ya lo hacen.
2. **Igual que los escenarios.** Cada escenario de los ciclos tendrá 3 casos (sección 6). Con 5 en la línea base, la referencia es un poco más estable que cada escenario, sin quitarles tiempo a los ciclos.
3. **El costo real.** Un caso tomó entre 24 y 76 minutos con corrección, más el codewalk, la ficha y la grabación. Con 24 tareas en total, cada caso de línea base es uno menos para los escenarios.
4. **El tipo de estudio.** Es un estudio exploratorio de casos múltiples: los resultados se describen y no se generalizan [23][24]. Con 5 o con 10 casos la conclusión sigue siendo una tendencia, no una causa.
5. **La herramienta.** Los 2 casos nuevos además prueban la grabación con Live Share antes de usarla en los ciclos. Se anota en la hoja Sesiones que cambió la herramienta.

Las tareas de línea base que no se usen (t16, t04, t14, t20 y t10) pasan a los escenarios en el mismo orden del sorteo, para no elegirlas a mano.

5 Roles definidos

Conductor y navegante no son lo mismo que junior y senior: son funciones, no niveles de experiencia [6][7]. Por eso la seniority se aplica a los agentes, donde sí se puede definir con su especificación.

Rol	Quién	Qué hace	Qué no hace
Conductor	Persona o agente	Escribe el código y explica en voz alta lo que hace	No decide solo si la tarea terminó
Navegante	Persona o agente	Revisa cada paso, piensa en los casos borde y vigila los criterios de aceptación	No escribe en el editor
Decisor	Siempre una persona	Acepta, corrige o rechaza lo que propone el agente, con su razón en la bitácora. Corre las ocultas	No delega esa decisión
Agente junior (implementador)	Agente	Escribe el código que se le pide y declara sus supuestos	No elige el enfoque ni da la tarea por terminada
Agente senior (mentor)	Agente	Revisa, hace preguntas y señala riesgos y casos borde. Responde con preguntas antes que con código	No escribe la solución

La idea de repartir el trabajo entre agentes con roles de un equipo real viene de ChatDev [15] y MetaGPT [16], donde los agentes hacen de programador, revisor o tester y cada uno revisa lo que entrega el otro.

6 Escenarios y casos

Cuatro escenarios aparte de la línea base, cada uno con 3 casos. Cada escenario responde una pregunta distinta frente a la línea base.

Escenario	Conduce	Navega	Qué pregunta responde	Tareas (orden del sorteo)
E0 Línea base	Persona	Persona	Cómo se hace hoy, sin IA	t06, t03, t12, t15, t07
E1 Persona conduce, agente navega	Persona	Agente (spec Copiloto v1)	Qué pasa si el agente reemplaza al compañero que revisa	t01 S, t23 M, t21 L
E2 Agente conduce, persona navega	Agente junior	Persona	La otra dirección: qué pasa si el agente escribe y la persona revisa. E1 frente a E2 es la comparación bidireccional	t02 S, t22 M, t19 M

E3 Tríada: dos personas más agente	Persona	Persona, con un agente compartido	Qué le suma el agente a una pareja humana. Es la comparación más directa con la línea base [14]	t16 S, t04 M, t14 M
E4 Agentes con seniority	Agente junior	Agente senior; la persona decide	Si repartir roles entre agentes da mejor código y deja a la persona pensando más	t09 S, t18 S, t11 M

Línea base	Ciclo 1	Entre ciclos	Ciclo 2	Total
5 casos, en pareja	E1 y E2: cada uno trabaja solo con el agente, en paralelo	Ajustar las specs con lo que pasó en la bitácora	E3 en pareja y E4 individual	17 casos medidos

Cómo salen estas tareas: E1 y E2 usan las 6 tareas que el sorteo ya asignó al ciclo 1. Cada uno hace de las dos direcciones: Ana Sofía t01 (E1), t02 (E2) y t23 (E1); Juan José t22 (E2), t21 (E1) y t19 (E2). E3 usa las primeras tareas de línea base que quedan libres, porque se comparan con la línea base. E4 usa las primeras del ciclo 2 del sorteo. Quedan de repuesto t20, t10, t24, t08, t17, t05 y t13.

Qué hay que preparar:

- **E1:** nada nuevo, usa la spec Copiloto v1.
- **E2:** una spec nueva para el agente junior, que escribe el código.
- **E3:** la misma spec v1, con un agente que ven los dos.
- **E4:** dos specs, senior y junior, y media página que diga cómo se pasan el trabajo, quién tiene la última palabra y qué pasa si no coinciden. El enunciado lo pide cuando hay varios agentes.

> SI NO ALCANZA EL TIEMPO
>
> Quiten E4, que es el que más preparación pide. Quedan 3 escenarios y 14 casos, y la pregunta bidireccional y la de la tríada se mantienen. Lo que no conviene es poner menos de 3 casos por escenario: con 1 o 2 no se ve ninguna tendencia.

7 Pensamiento crítico

El riesgo es aceptar lo que dice la IA sin pensarlo. Quienes más confían en la IA reportan pensar menos críticamente [17], y en programación algunos estudiantes terminan con una falsa sensación de que entienden [18][19].

Cómo medirlo sin métricas nuevas:

- **M4 Codewalk por escenario:** si con el agente baja la nota, la persona entiende menos lo que entregó.
- **Porcentaje de propuestas aceptadas sin modificar**, de la columna que ya tiene la bitácora. Si casi todo se acepta sin cambios, es una señal de alerta.
- **Correcciones y rechazos con su razón:** muestran que alguien revisó.

Reglas para reducirlo, en todos los escenarios con agente:

1. **Pensar antes de preguntar:** 2 minutos para anotar el enfoque propio antes de abrir el agente.
2. **Explicar sin mirar:** no se acepta una línea que no puedan explicar sin el agente. Ya está en la spec v1.
3. **Un agente que pregunta:** el agente senior responde primero con preguntas y no con código.
4. **La última palabra es humana:** las ocultas las corre siempre una persona.
5. **Alternar quién conduce** entre tareas, para que nadie se quede solo revisando.

8 Lo que tienen que decidir

1. **Editor:** VS Code con Live Share o PyCharm con Code With Me.
2. **Escenarios:** los 4 (17 casos) o sin E4 (14 casos).
3. **Acta:** escribir lo que dijo el profe sobre la cantidad flexible de casos y guardarla en esta carpeta.
4. **Specs:** si siguen, preparo las nuevas (junior, senior y su articulación) y las agrego a la bitácora con una columna *Escenario*.

9 Referencias

Todas son revisadas por pares, salvo los libros y la documentación de herramientas, que se indican, y [22], que es un preprint.

1. Beck, K. (1999). *Extreme programming explained: Embrace change*. Addison-Wesley. (Libro.) https://www.oreilly.com/library/view/extreme-programming-explained/0201616416
2. Williams, L., & Kessler, R. (2002). *Pair programming illuminated*. Addison-Wesley. (Libro.) https://openlibrary.org/books/OL22018567M/Pair_programming_illuminated
3. Williams, L., Kessler, R. R., Cunningham, W., & Jeffries, R. (2000). Strengthening the case for pair programming. *IEEE Software, 17*(4), 19–25. https://doi.org/10.1109/52.854064 · PDF: https://collaboration.csc.ncsu.edu/laurie/Papers/ieeeSoftware
4. Hannay, J. E., Dybå, T., Arisholm, E., & Sjøberg, D. I. K. (2009). The effectiveness of pair programming: A meta-analysis. *Information and Software Technology, 51*(7), 1110–1122. https://doi.org/10.1016/j.infsof.2009.02.001
5. Arisholm, E., Gallis, H., Dybå, T., & Sjøberg, D. I. K. (2007). Evaluating pair programming with respect to system complexity and programmer expertise. *IEEE Transactions on Software Engineering, 33*(2), 65–86. https://doi.org/10.1109/TSE.2007.17
6. Bryant, S., Romero, P., & du Boulay, B. (2008). Pair programming and the mysterious role of the navigator. *International Journal of Human-Computer Studies, 66*(7), 519–529. https://doi.org/10.1016/j.ijhcs.2007.03.005
7. Chong, J., & Hurlbutt, T. (2007). The social dynamics of pair programming. *ICSE 2007*, 354–363. https://doi.org/10.1109/ICSE.2007.87
8. Plonka, L., Sharp, H., van der Linden, J., & Dittrich, Y. (2015). Knowledge transfer in pair programming: An in-depth analysis. *International Journal of Human-Computer Studies, 73*, 66–78. https://doi.org/10.1016/j.ijhcs.2014.09.001
9. Lui, K. M., & Chan, K. C. C. (2006). Pair programming productivity: Novice–novice vs. expert–expert. *International Journal of Human-Computer Studies, 64*(9), 915–925. https://research.polyu.edu.hk/en/publications/pair-programming-productivity-novice-novice-vs-expert-expert/
10. Zieris, F., & Prechelt, L. (2016). Observations on knowledge transfer of professional software developers during pair programming. *ICSE 2016 Companion (SEIP)*, 242–250. https://doi.org/10.1145/2889160.2889249

11. Salinger, S., & Prechelt, L. (2013). *Understanding pair programming: The base layer*. (Libro académico, basado en una tesis doctoral.) https://mi.fu-berlin.de/w/SE/PairProgrammingHome
12. Welter, A., Schneider, N., Dick, T., Weis, K., Tinnes, C., & Wyrich, M. (2025). An empirical study of knowledge transfer in AI pair programming. *ASE 2025*, 166–177. https://doi.org/10.1109/ASE63991.2025.00022
13. Lyu, W., Wang, Y., Sun, Y., & Zhang, Y. (2025). Will your next pair programming partner be human? An empirical evaluation of generative AI as a collaborative teammate in a semester-long classroom setting. *L@S '25*, 83–94. https://arxiv.org/abs/2505.08119
14. Daryanto, T., et al. (2026). Human-human-AI triadic programming: Uncovering the role of AI agent and the value of human partner in collaborative learning. *CHI '26*. https://arxiv.org/abs/2601.12134
15. Qian, C., et al. (2024). ChatDev: Communicative agents for software development. *ACL 2024*. https://arxiv.org/abs/2307.07924
16. Hong, S., et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. *ICLR 2024*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html
17. Lee, H.-P., Sarkar, A., Tankelevitch, L., Drosos, I., Rintel, S., Banks, R., & Wilson, N. (2025). The impact of generative AI on critical thinking. *CHI 2025*. https://doi.org/10.1145/3706598.3713778
18. Prather, J., et al. (2024). The widening gap: The benefits and harms of generative AI for novice programmers. *ICER 2024*. https://arxiv.org/abs/2405.17739
19. Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI code generators on supporting novice learners in introductory programming. *CHI 2023*, 1–23. https://doi.org/10.1145/3544548.3580919
20. Microsoft. (s. f.). Coedit, follow, and focus in a collaboration session in Visual Studio Code. (Documentación.) https://learn.microsoft.com/en-us/visualstudio/liveshare/use/coedit-follow-focus-visual-studio-code
21. JetBrains. (2025). Code With Me. *PyCharm Help*. (Documentación.) https://www.jetbrains.com/help/pycharm/2025.2/code-with-me.html
22. CodeWatcher: IDE telemetry data extraction tool for understanding coding interactions with LLMs. (2025). arXiv:2510.11536. (Preprint.) https://arxiv.org/abs/2510.11536
23. Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer. https://doi.org/10.1007/978-3-642-29044-2
24. Kitchenham, B., Madeyski, L., Budgen, D., Keung, J., Brereton, P., Charters, S., Gibbs, S., & Pohthong, A. (2017). Robust statistical methods for empirical software engineering. *Empirical Software Engineering, 22*(2), 579–630. https://doi.org/10.1007/s10664-016-9437-5